

XJTLU Entrepreneur College (Taicang) Cover Sheet

Module code and Title	DTS111TC Computer Architecture and Operating Systems		
School Title	School of AI and Advanced Computing		
Assignment Title	Coursework 1		
Submission Deadline	20/April/2026		
Final Word Count	N/A		
If you agree to let the university use your work anonymously for teaching and learning purposes, please type "yes" here.			yes

I certify that I have read and understood the University's Policy for dealing with Plagiarism, Collusion and the Fabrication of Data (available on Learning Mall Online). With reference to this policy, I certify that:

- ⑩ My work does not contain any instances of plagiarism and/or collusion.
My work does not contain any fabricated data.

By uploading my assignment onto Learning Mall Online, I formally declare that all of the above information is true to the best of my knowledge and belief.

Scoring – For Tutor Use					
Student ID			2471555		

Stage of Marking	Marker Code	Learning Outcomes Achieved (F/P/M/D) (please modify as appropriate)			Final Score
		A	B	C	
1 st Marker – red pen					
Moderation – green pen	IM Initials	The original mark has been accepted by the moderator (please circle as appropriate):			Y / N
		Data entry and score calculation have been checked by another tutor (please circle):			Y
2 nd Marker if needed – green pen					
For Academic Office Use		Possible Academic Infringement (please tick as appropriate)			
Date Received	Days late	Late Penalty	☐ Category A ☐ Category B ☐ Category C ☐ Category D ☐ Category E		Total Academic Infringement Penalty (A,B, C, D, E, Please modify where necessary) _____

T1 Data Format (20 marks)

T1-4 (Program Evidence): Execute the computation, print the outputs, and fill the results into the provided table in your report. Paste a clear screenshot that captures both your code and the output.

Reuslt Table	<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>[[150. 150. 150.] [150. 130. 132.5] [100. 100. 100.]]</td><td>11000000.0</td><td>0.4999999994</td></tr></table>	A	B	C	[[150. 150. 150.] [150. 130. 132.5] [100. 100. 100.]]	11000000.0	0.4999999994
A	B	C					
[[150. 150. 150.] [150. 130. 132.5] [100. 100. 100.]]	11000000.0	0.4999999994					
Screenshot	CW1_Codes_v2 >  CW_A_T1.py <pre>4 5 #Write your code here and print the results 6 #T1.1 7 image1 = data_1.astype(np.uint16) 8 image2 = data_2.astype(np.uint16) 9 result_1 = (image1 + image2) / 2.0 10 print("result_1: ") 11 print(result_1) 12 #T1.2 13 total_loss = np.float64(data_3) 14 record = np.float64(0.0) 15 for update in data_4: 16 y = np.float64(update) - record 17 t = total_loss + y 18 record = (t - total_loss) - y 19 total_loss = t 20 print("\nresult_2 :") 21 print(total_loss) 22 #T1.3 23 x = np.float64(data_5) 24 result_3 = 2.0 * (np.sin(x / 2.0) ** 2) / (x ** 2) 25 print("\nresult_3 :") 26 print(result_3) 27</pre> 问题 输出 调试控制台 终端 端口 ● (base) peng_jiang@Peng-JiangdeMacBook-Air CW1_codes_v2 % pwd ● (base) peng_jiang@Peng-JiangdeMacBook-Air CW1_codes_v2 % python CW_A_T1.py <pre>result_1: [[150. 150. 150.] [150. 130. 132.5] [100. 100. 100.]] result_2 : 11000000.0 result_3 : 0.4999999994 ○ (base) peng_jiang@Peng-JiangdeMacBook-Air CW1_codes_v2 %</pre>						

T1-5 (Analysis): For each of the three computational tasks above, answer the following in your report: What will a naive calculation cause and why would it happen? How did you modify your code or formula to ensure numerical accuracy?

Analysis	T1.1: A naive calculation may cause integer overflow if the two image matrices are stored in 8-bit unsigned integer format. When two large pixel values are added directly, the sum may exceed 255 and overflow, which would produce an incorrect average. To avoid this issue, I first converted both matrices to uint16 type before performing the addition. This ensures that the sum of data_1 and data_2 are stored safely and the final 3×3 average matrix is numerically correct. T1.2: Naive accumulation may cause floating-point precision loss, because a very large base value is repeatedly updated by millions of tiny increments. In ordinary floating-point addition, some very small updates may be rounded away when added to a much larger number. To improve numerical accuracy, I used float64 together with Kahan summation. This method introduces a compensation variable to track the low-order bits lost during repeated additions, thereby reducing the accumulation of rounding errors. T1.3: A naive calculation of $(1 - \cos x) / x^2$ may cause catastrophic cancellation when x is very small, because $\cos x$ is extremely close to 1. Subtracting two nearly equal floating-point numbers causes a significant loss of precision. To ensure numerical stability, I rewrote the numerator using the trigonometric identity. $1 - \cos x = 2(\sin(x / 2))^2$. So, the new formula is $(2(\sin(x / 2))^2) / x^2$. This form is much more stable for small values of x, so it produces a more accurate result.
----------	---

T2. MARIE Assembly Program – (30 marks)

T2-2 (Program Evidence): Run your program in the MARIE.js simulator. Paste a clear screenshot into your report that captures the Assembly Code, Memory Map, and Output Log.

Program Evidence

1
2 Load Num1
3 Add Num2
4 Store Result
5 Output
6 Halt
7
8 Num1, DEC 2
9 Num2, DEC 3
10 Result, DEC 0
11 END

Machine halted normally.

AC 0005 IR 7000 MAR 004 MBR 7000 PC 005 IN 0000 OUT 0005

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
000	1005	3006	2007	6000	7000	0002	0003	0005	0000	0000	0000	0000	0000	0000	0000	0000
010	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
020	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
030	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
040	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
050	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
060	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
070	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Output log
Hexadecimal
0005

RTL log
MAR ← PC
MBR ← M[MAR]
IR ← MBR
PC ← PC + 1
Decoded opcode 0x1 in IR[15-12] as Load
MAR ← IR[11-0]
MBR ← M[MAR]
AC ← MBR

MAR ← PC
MBR ← M[MAR]
IR ← MBR
PC ← PC + 1
Decoded opcode 0x3 in IR[15-12] as Add
MAR ← IR[11-0]

Watch list

Label	Address	HEX	DEC	OCT	BIN
Num1	→ 005	0002	2	2	10
Num2	→ 006	0003	3	3	11

Inputs

Display

T2-3 (RTL Analysis): Refer to your RTL Log to state the total number of instruction cycles performed. List each instruction executed along with its memory address. Your answers must include specific data from your own simulation.

Answer	The program performed a total of 5 instruction cycles, because five instructions were executed from start to halt. The instructions executed in order were: 1. Load Num1 – Address 000 2. Add Num2 – Address 001 3. Store Result – Address 002 4. Output – Address 003 5. Halt – Address 004 In the simulation, the result was successfully stored and the output log displayed 0005, so this is correct execution.
--------	--

T2-4: Every instruction starts with a repeated sequence of micro-operations. Write the RTL notation for this repeated sequence. Explain the purpose of this sequence by referencing the specific PC and MAR values used during the first two instructions of your program.

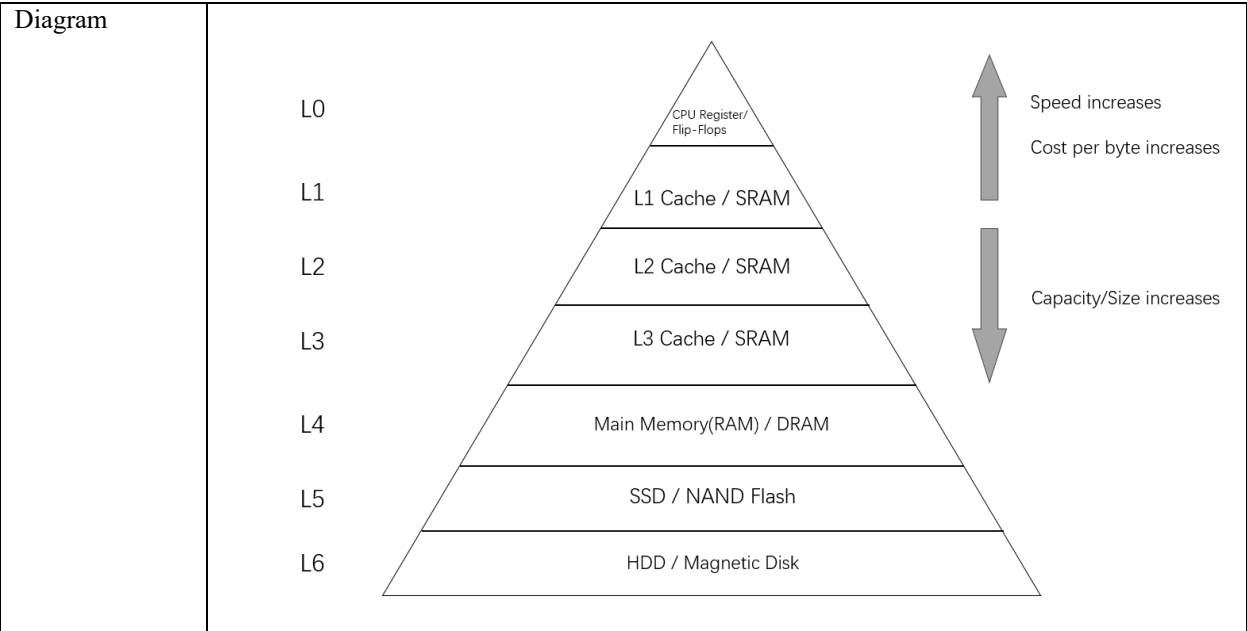
RTL	MAR <-- PC MBR <-- M[MAR] IR <-- MBR PC <-- PC + 1 MAR <-- IR[11-0]
-----	---

Analysis	This repeated sequence is the fetch/decode phase. Its purpose is to fetch the next instruction from memory, move it into the instruction register, advance the program counter, and extract the operand address for execution. For the first instruction, the initial value of PC is 000, so the CPU first copies 000 into MAR. It then fetches the instruction stored at address 000, which is Load Num1, and increments PC to 001. After decoding, the operand address 005 is placed into MAR. For the second instruction, PC is 001 now, so the CPU copies 001 into MAR, fetches the instruction at address 001, which is Add Num2, and increments PC to 002. After decoding, the operand address 006 is placed into MAR. Hence, repeated sequence allows the CPU to locate, fetch, and prepare each instruction before execution.
----------	---

T2-5 (ADD Instruction Analysis): Analyze the execution phase of your ADD instruction. List the unique RTL steps that occur after the fetch/decode phase. Identify the specific hex values being moved into the MBR and AC during these steps, and explain the internal logic of how the AC reaches its final value.

RTL	MBR $\leftarrow$ M[MAR] AC $\leftarrow$ AC + MBR
Hex Value	Operand address in MAR: 006 Value moved into MBR: 0003 Value in AC before addition: 0002 Final value in AC: 0005
Analysis	After the fetch/decode phase of the Add Num2 instruction, the operand address 006 has already been placed into MAR. The CPU then reads the content of memory location 006 into MBR. From the memory map, the value stored at address 006 is 0003, so MBR = 0003. Before the Add instruction executes, the accumulator already contains the value loaded from Num1, which is 0002. The ALU then performs the addition of 0002 + 0003, so the final value in the accumulator becomes 0005. This value is later stored in Result and displayed in the output log. The CU is a Control Unit, which coordinates the fetch, decode, and execution steps by sending control signals to the registers, memory, and ALU. During the Add instruction, the CU ensures the operand is fetched from memory into the MBR and that the ALU adds it to the current value in the AC.

T3. Memory Hierarchy – (20 marks)



T4. Evaluating CPU Scheduling Policies – (20 marks)

T4-1 (Calculation): Compute the Average Turnaround Time for Case A (FIFO) manually and show your work in the report.

Calculation	The jobs are executed in the order A --> B --> C for FIFO. Job A runs from 0 to 100, so its turnaround time is 100. Job B runs from 100 to 110, so its turnaround time is 110. Job C runs from 110 to 120, so its turnaround time is 120. Therefore, the Average Turnaround Time is $(100 + 110 + 120) / 3 = 110$.
-------------	---

T4-2 (Simulation): Run the scheduler.py simulator for three cases: Case A (FIFO), Case B (SJF), and Case C (RR, Quantum = 1).

Results	<table><tr><th>Case</th><th>Policy</th><th>Parameters</th><th>Avg Turnaround</th><th>Avg Response</th></tr><tr><td>A</td><td>FIFO</td><td>N/A</td><td>110</td><td>70</td></tr><tr><td>B</td><td>SJF</td><td>N/A</td><td>50</td><td>10</td></tr><tr><td>C</td><td>RR</td><td>Quantum = 1</td><td>59.67</td><td>1</td></tr></table>	Case	Policy	Parameters	Avg Turnaround	Avg Response	A	FIFO	N/A	110	70	B	SJF	N/A	50	10	C	RR	Quantum = 1	59.67	1
Case	Policy	Parameters	Avg Turnaround	Avg Response																	
A	FIFO	N/A	110	70																	
B	SJF	N/A	50	10																	
C	RR	Quantum = 1	59.67	1																	
Program Evidence	Case A: <pre>• (base) peng_jiang@Peng-JiangdeMacBook-Air CW1_Codes_v2 % pwd /Users/peng_jiang/Downloads/OS/CW1_Files-20260415/CW1_Codes_v2 • (base) peng_jiang@Peng-JiangdeMacBook-Air CW1_Codes_v2 % python scheduler.py -p FIFO -l 100,10,10 -c ARG policy FIFO ARG jlist 100,10,10 Here is the job list, with the run time of each job: Job 0 (length = 100.0) Job 1 (length = 10.0) Job 2 (length = 10.0) ** Solutions ** Execution trace: [time 0] Run job 0 for 100.00 secs (DONE at 100.00) [time 100] Run job 1 for 10.00 secs (DONE at 110.00) [time 110] Run job 2 for 10.00 secs (DONE at 120.00) Final statistics: Job 0 — Response: 0.00 Turnaround 100.00 Wait 0.00 Job 1 — Response: 100.00 Turnaround 110.00 Wait 100.00 Job 2 — Response: 110.00 Turnaround 120.00 Wait 110.00 Average — Response: 70.00 Turnaround 110.00 Wait 70.00 • (base) peng_jiang@Peng-JiangdeMacBook-Air CW1_Codes_v2 %</pre>																				

Case B:

```
(base) peng_jiang@Peng-JiangdeMacBook-Air CW1_Codes_v2 % python scheduler.py -p SJF -l 100,10,10 -c
ARG policy SJF
ARG jlist 100,10,10

Here is the job list, with the run time of each job:
Job 0 ( length = 100.0 )
Job 1 ( length = 10.0 )
Job 2 ( length = 10.0 )

** Solutions **

Execution trace:
[ time 0 ] Run job 1 for 10.00 secs ( DONE at 10.00 )
[ time 10 ] Run job 2 for 10.00 secs ( DONE at 20.00 )
[ time 20 ] Run job 0 for 100.00 secs ( DONE at 120.00 )

Final statistics:
Job 1 -- Response: 0.00 Turnaround 10.00 Wait 0.00
Job 2 -- Response: 10.00 Turnaround 20.00 Wait 10.00
Job 0 -- Response: 20.00 Turnaround 120.00 Wait 20.00

Average -- Response: 10.00 Turnaround 50.00 Wait 10.00

(base) peng_jiang@Peng-JiangdeMacBook-Air CW1_Codes_v2 %
```

Case C:

```
(base) peng_jiang@Peng-JiangdeMacBook-Air CW1_Codes_v2 % python scheduler.py -p RR -q 1 -l 100,10,10 -c
[ time 78 ] Run job 0 for 1.00 secs
[ time 79 ] Run job 0 for 1.00 secs
[ time 80 ] Run job 0 for 1.00 secs
[ time 81 ] Run job 0 for 1.00 secs
[ time 82 ] Run job 0 for 1.00 secs
[ time 83 ] Run job 0 for 1.00 secs
[ time 84 ] Run job 0 for 1.00 secs
[ time 85 ] Run job 0 for 1.00 secs
[ time 86 ] Run job 0 for 1.00 secs
[ time 87 ] Run job 0 for 1.00 secs
[ time 88 ] Run job 0 for 1.00 secs
[ time 89 ] Run job 0 for 1.00 secs
[ time 90 ] Run job 0 for 1.00 secs
[ time 91 ] Run job 0 for 1.00 secs
[ time 92 ] Run job 0 for 1.00 secs
[ time 93 ] Run job 0 for 1.00 secs
[ time 94 ] Run job 0 for 1.00 secs
[ time 95 ] Run job 0 for 1.00 secs
[ time 96 ] Run job 0 for 1.00 secs
[ time 97 ] Run job 0 for 1.00 secs
[ time 98 ] Run job 0 for 1.00 secs
[ time 99 ] Run job 0 for 1.00 secs
[ time 100 ] Run job 0 for 1.00 secs
[ time 101 ] Run job 0 for 1.00 secs
[ time 102 ] Run job 0 for 1.00 secs
[ time 103 ] Run job 0 for 1.00 secs
[ time 104 ] Run job 0 for 1.00 secs
[ time 105 ] Run job 0 for 1.00 secs
[ time 106 ] Run job 0 for 1.00 secs
[ time 107 ] Run job 0 for 1.00 secs
[ time 108 ] Run job 0 for 1.00 secs
[ time 109 ] Run job 0 for 1.00 secs
[ time 110 ] Run job 0 for 1.00 secs
[ time 111 ] Run job 0 for 1.00 secs
[ time 112 ] Run job 0 for 1.00 secs
[ time 113 ] Run job 0 for 1.00 secs
[ time 114 ] Run job 0 for 1.00 secs
[ time 115 ] Run job 0 for 1.00 secs
[ time 116 ] Run job 0 for 1.00 secs
[ time 117 ] Run job 0 for 1.00 secs
[ time 118 ] Run job 0 for 1.00 secs
[ time 119 ] Run job 0 for 1.00 secs ( DONE at 120.00 )

Final statistics:
Job 0 -- Response: 0.00 Turnaround 120.00 Wait 20.00
Job 1 -- Response: 1.00 Turnaround 29.00 Wait 19.00
Job 2 -- Response: 2.00 Turnaround 30.00 Wait 20.00

Average -- Response: 1.00 Turnaround 59.67 Wait 19.67

(base) peng_jiang@Peng-JiangdeMacBook-Air CW1_Codes_v2 %
```

T4-3 (Observation & Analysis): Comparing Case A and Case B, explain the phenomenon you observe regarding the order of jobs and the turnaround time.

Observation & Analysis	Comparing Case A and Case B, the main difference is the execution order of the jobs. In FIFO, the longest job A runs first, so the two short job B and job C must wait until A finishes. This causes both short jobs to have very large turnaround times, which increases the overall average turnaround time. In SJF, it schedules the two short jobs first. Since B and C only require 10 units each, they finish much earlier, so the average turnaround time drops significantly from 110.00 to 50.00. This demonstrates that job order has a strong effect on performance, and that running short jobs first is usually more efficient for reducing average turnaround time.
------------------------	---

T4-4 (Observation & Analysis): Based on the results of Case C (Round Robin), explain the fundamental trade-off between RR and SJF/STCF. Furthermore, discuss what happens if the time slice is set too small.

Observation & Analysis	The fundamental trade-off between Round Robin and SJF/STCF is fairness versus turnaround efficiency. RR gives each job a chance to run quickly, so it provides very good response time and improves interactivity. In this case, the average response time of RR is only 1.00, which is much better than FIFO and SJF. However, RR does not minimize turnaround time as effectively as SJF or STCF, because it repeatedly rotates among jobs instead of allowing short jobs to finish immediately. As a result, its average turnaround time is usually higher than the optimal short-job-first style policies. If the time slice is set too small, the CPU will switch between jobs too frequently. This increases scheduling and context-switch overhead, which wastes CPU time and reduces overall efficiency. Therefore, a very small quantum improves responsiveness, but may hurt total system performance.
------------------------	---